

Drone airspace: the strongest build bet

Training data, available sources, and three problem statements for Opus Build Day.

Recommendation

Build an **Adversarial Flight Lab**: the system finds landing conditions a controller fails, proposes a bounded change, and tests whether that change works on unseen flights.

What “training data” means here

The first version does **not** need a large labeled dataset or retraining of Opus. Each simulated landing is an experiment: record aircraft state, six-degree-of-freedom deck motion, weather, controller action, and outcome. A fixed evaluator scores touchdown speed, pad error, aborts, and failures. Opus proposes policy changes; the simulator accepts or rejects them.

DATA	PURPOSE / LIMIT	SOURCE
Synthetic attempts	Generate wave and flight conditions in a simulator. Use for the improvement loop; hold back separate seeds and conditions for evaluation.	Gazebo wave simulation
Ocean conditions	Use historical wave spectra and weather to choose more plausible test conditions. These are not deck-motion recordings.	NOAA NDBC history
Ship motion	Use recorded ship IMU traces to check prediction and realism; inspect coverage and license before use.	RMA maritime landing dataset
Aircraft logs	Use public PX4 logs to exercise log ingestion and check general flight dynamics. Most are not paired ship landings.	PX4 public logs
Own operations	For a real product, collect synchronized deck pose, aircraft pose, control inputs, wind, and outcomes from the intended vessel and aircraft.	Not publicly replaceable

If we later train a neural wave predictor or reinforcement-learning controller, that requires many more simulation runs plus validation against vessel-specific recordings. Synthetic success alone is not evidence of real-world landing performance.

Three problems worth building for

Ranked for a working, understandable hackathon demonstration.

01 Adversarial Flight Lab | Recommended

Problem: How can a drone team discover where a controller will fail, then improve it before encountering that situation in flight?

Build: Opus creates hard but plausible simulated conditions, diagnoses the failure, proposes a bounded controller change, and runs it against unseen cases. Show baseline and improved versions side by side.

Judging: Strong on New Capability and Clarity of Demo if the complete loop works. Useful to teams building many kinds of drone controllers.

02 Offline Landing Envelope

Problem: Under which deck motions and levels of uncertainty should a drone attempt, wait, or abort a landing?

Build: Give a shipboard operator a live, interpretable landing window based on deck prediction and aircraft state. Run the flight decision without internet.

Judging: Strong direct operational value. Harder to prove forecast quality and real-world reliability in one day.

03 Airspace Conflict Simulator

Problem: When several drone missions overlap and communications are unreliable, which plans remain conflict-free?

Build: Show mission intents on a map, find a conflict, and propose a revised plan that passes scenario checks.

Judging: Broader audience and clear visuals. Drone traffic management already exists, so the novel contribution must be specific and demonstrated.

The 90-second proof

Show a baseline failure, the condition Opus discovered, its proposed change, and a held-out comparison across many runs. Use the same aircraft and evaluation rules for both versions. A single successful landing is not enough evidence.

Suggested pitch line

“Drone teams test the conditions they expect. Our system finds the conditions they missed, improves a candidate controller in simulation, and shows whether the improvement survives unseen flights.”

Context: [FAA UAS Traffic Management](#) shows why the third idea needs a clearly distinct contribution. Judge interest cannot be guaranteed; the recommendation follows the event rubric supplied: New Capability, It Works, Keep or Share, and Clarity of Demo.